ASP.NET Core uses small modular components for each distinct feature. This approach allows individual features to evolve separately, with only a loose coupling to others, and it's generally considered to be good design practice. The downside to this approach is that it places the burden on the consumer of a feature to instantiate it correctly. Within your application, these modular components are exposed as one or more *services* that are used by the application.

DEFINITION Within the context of ASP.Net Core, *service* refers to any class that provides functionality to an application. Services could be classes exposed by a library or code you've written for your application.

In an e-commerce app, for example, you might have a `TaxCalculator` that calculates the tax due on a particular product, taking into account the user's location in the world. Or you might have a `ShippingCostService` that calculates the cost of shipping to a user's location. A third service, `OrderTotalCalculator`, might use both of these services to work out the total price the user must pay for an order. Each service provides a small piece of independent functionality, but you can combine them to create a complete application. This design methodology scenario is known as the *single-responsibility principle.*

DEFINITION The *single-responsibility principle* (SRP) states that every class should be responsible for only a single piece of functionality; it should need to change only if that required functionality changes. SRP is one of the five main design principles promoted by Robert C. Martin in *Agile Software Development, Principles, Patterns, and Practices* (Pearson, 2013).

`OrderTotalCalculator` needs access to an instance of `ShippingCostService` and `TaxCalculator`. A naive approach to this problem is to use the `new` keyword and create an instance of a service whenever you need it. Unfortunately, this approach tightly couples your code to the specific implementation you're using and can undo all the good you achieved by modularizing the features in the first place. In some cases, it may break the